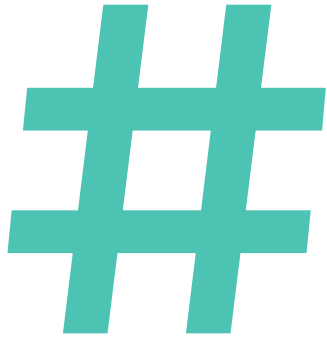




Hash Table

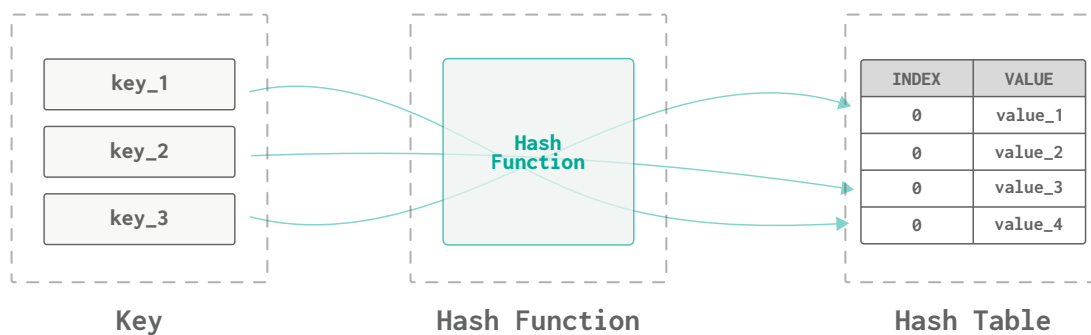
Step 01 What is a Hash Table

Imagine you have a list of phone numbers that you need to store and manage efficiently. By implementing a hash table in this scenario, you can create a seamless association between each person's name and their corresponding phone number. In this context, the person's name serves as the key, while the phone number acts as the value.

A practical analogy for this concept is a dictionary. Just as a dictionary allows you to look up a word (the key) to find its definition (the value), a hash table enables you to quickly search for a person's name to retrieve their phone number.

Hash table is a type of data structure in which the index value of the data element is generated from a hash function $\text{hash} = \text{key} \% \text{array_size}$.

```
01 public int hashFunction(int key, int arrSize) {
02     return key % arrSize;
03 }
```



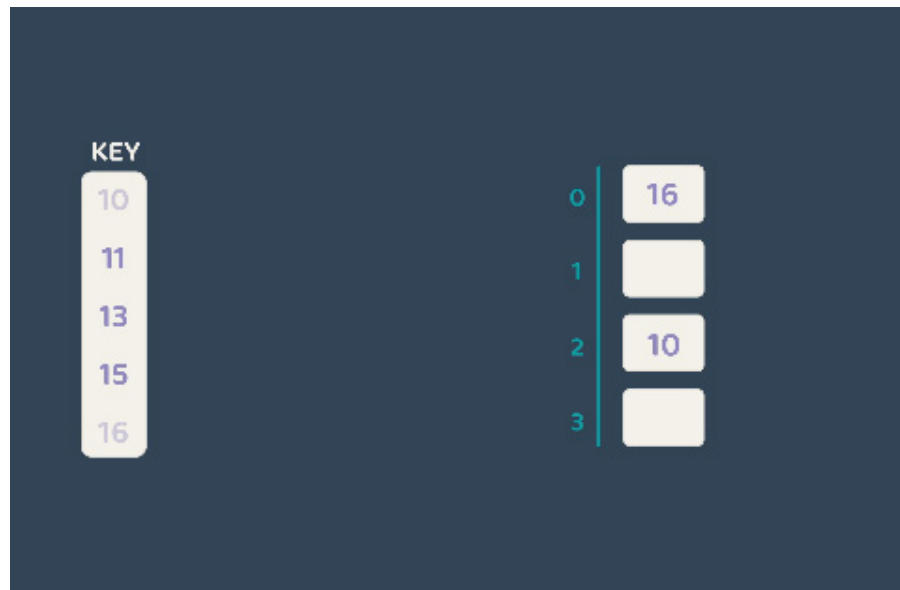
Step 02 Collision Handling

Collision Handling happens when two keys are hashed to the same index in a hash table. Collisions are a problem because every slot in a hash table is supposed to store a single element, to handle collisions there are various techniques to handle such as chaining and open addressing.

For example, consider a hash table with a size of 10, where we want to insert the keys 12 and 22. If we use a simple hash function that computes the index as $\text{key} \% \text{table_size}$, both keys would hash to the same index. Since both keys hash to index 2, a collision occurs.

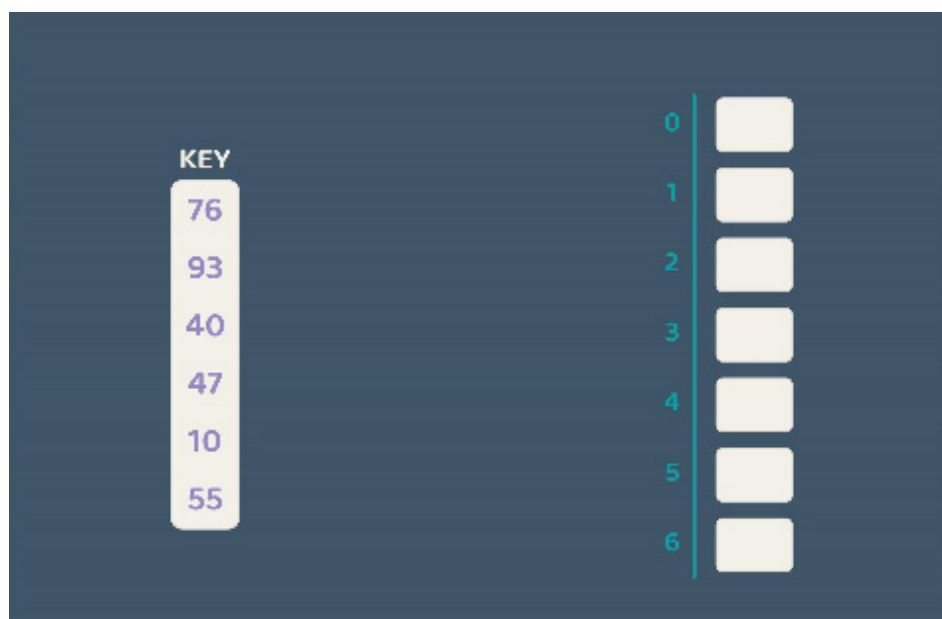
01 Chaining

Each bucket in the hash table contains a linked list to store multiple key-value pairs that hash to the same index so we can handle a random number of collisions using an array of linked lists, each index has its own linked list.



02 Open Address

Stores all key-value pairs directly in the hash table itself, without using separate data structures like linked lists. when a collision occurs, open addressing involves probing the table to find an alternative index for the colliding element (checking the next available slot).



Step 03 Implementation

```

01     private static class Node {
02         int key;
03         int value;
04         Node next;
05         public Node(int key, int value) {
06             this.key = key;
07             this.value = value;
08         }
09     }

```

CODE Create Node class.

```

01     public class HashTable {
02         private int size;
03         private Node[] table;
04         public HashTable(int size) {
05             this.size = size;
06             table = new Node[size];
07         }

```

CODE Create a hash table using the linked lists collision chaining technique.

```

01     public void put(int key, int value) {
02         int index = hash(key);
03         Node node = table[index];
04         if (node == null) {
05             table[index] = new Node(key, value);
06         } else {
07             while (node.next != null) {
08                 node = node.next;
09             }
10             node.next = new Node(key, value);
11         }
12     }

```

CODE The put method inserts a new key-value pair into the hash table.

```

01     public int get(int key) {
02         int index = hash(key);
03         Node node = table[index];
04         while (node != null) {
05             if (node.key == key) {
06                 return node.value;
07             }
08             node = node.next;
09         }
10         return -1;
11         // not found
12     }

```

CODE The get method retrieves the value associated with a given key.

```

01     private int hash(int key) {
02         return key % size;
03     }

```

CODE The hash method calculates the index of the table based on the key.

```

01     public static void main(String[] args) {
02         HashTable hashTable = new HashTable(10);
03
04         hashTable.put(1, 10);
05         hashTable.put(2, 20);
06         hashTable.put(11, 30);
07         hashTable.put(12, 40);
08
09
10         System.out.println("Get value for key 1: " + hashTable.get(1));
11         System.out.println("Get value for key 2: " + hashTable.get(2));
12         System.out.println("Get value for key 11: " + hashTable.get(11));
13         System.out.println("Get value for key 12: " + hashTable.get(12));
14         System.out.println("Get value for key 3: " + hashTable.get(3));
15
16
17     }
18 }

```

CODE Main method.

OUTPUT

Get value for key 1: 10

Get value for key 2: 20

Get value for key 11: 30

Get value for key 12: 40

Get value for key 3: -1